# Developer Guide

**Date:** 2026-06-12

Adobe

# Table of Contents

ISE Boilerplate - Developer Guide ..... 3
- Quick Reference ..... 3
- Architecture Overview ..... 3
- Local Development Setup ..... 6
- Design System ..... 7
- Blocks Reference ..... 9
- Universal Editor Models ..... 11
- Common Development Tasks ..... 12
- Environments ..... 12
- Git and Release Workflow ..... 13
- Troubleshooting ..... 13
- Resources ..... 14

# ISE Boilerplate - Developer Guide

A complete technical guide for developers maintaining and extending this AEM Edge Delivery Services project. The project is built on the AEM Block Collection / aem-boilerplate, authored through Document Authoring (DA) and Universal Editor (UE), with no build step and zero runtime dependencies.

## Quick Reference

| Resource | URL |
|---|---|
| Code Repository | github.com/aemdemos/ise-boilerplate |
| Preview | https://main--ise-boilerplate--aemdemos.aem.page/ |
| Live | https://main--ise-boilerplate--aemdemos.aem.live/ |
| Feature Branch | https://{branch}--ise-boilerplate--aemdemos.aem.page/ |
| Local Dev | http://localhost:3000 |
| Config Service Org | aemdemos |

## Architecture Overview

### Tech Stack

- Vanilla JavaScript (ES6+ native modules) — no transpiling, no bundler
- Native CSS3 with custom properties (design tokens) — no preprocessors or frameworks
- No build step for runtime code — files are served directly to the browser
- Zero production dependencies (devDependencies only, for linting/testing/release)
- Content authored in Document Authoring (da.live) and edited in Universal Editor
- Node.js 24 for local tooling; npm only

### Project Structure

```
├── blocks/             # 20 blocks, each {name}/{name}.js + {name}/{name}.css
├── scripts/
│   ├── aem.js          # Core AEM library (DO NOT MODIFY)
│   ├── scripts.js      # Page decoration entry point + E-L-D orchestration
│   ├── delayed.js      # Delayed-phase functionality (analytics/martech)
│   ├── utils.js        # Shared helpers (YouTube/Vimeo embed builders)
│   ├── slider.js       # Shared carousel/slider engine
│   ├── dompurify.min.js     # HTML sanitizer (lazy-loaded on demand)
│   ├── editor-support.js     # Universal Editor live-editing support
│   └── editor-support-rte.js # Rich-text editing support
├── styles/
│   ├── styles.css      # LCP-critical global styles + design tokens
│   ├── lazy-styles.css # Below-the-fold global styles
│   └── fonts.css       # @font-face declarations (Roboto)
├── ue/
│   ├── models/         # SOURCE of truth for UE model JSON (see "UE Models")
```

```
│   │   └── blocks/     # Per-block component definition/model/filter fragments
│   └── scripts/        # ue.js + ue-utils.js (UE-only runtime support)
├── icons/              # SVG icons (search.svg)
├── fonts/              # Roboto woff2 web fonts
├── tools/              # Sidekick + quick-edit tooling
├── tests/              # Accessibility tests (Playwright + axe-core)
├── component-definition.json # GENERATED — merged from ue/models/
├── component-models.json     # GENERATED — merged from ue/models/
├── component-filters.json    # GENERATED — merged from ue/models/
├── head.html           # Global <head> content (DO NOT MODIFY)
└── 404.html            # Custom error page
```

## Three-Phase Loading (Eager - Lazy - Delayed)

AEM Edge Delivery Services uses a strict Eager-Lazy-Delayed (E-L-D) loading strategy to achieve a Lighthouse score of 100. The orchestration lives in `scripts/scripts.js` via `loadEager()`, `loadLazy()`, and `loadDelayed()`, all driven by `loadPage()`.

| Phase | Function | What Loads | Performance Impact |
|---|---|---|---|
| **Eager** | `loadEager(doc)` | `styles/styles.css`, decorates `<main>` (`decorateMain`), loads the first section, waits for the first image (LCP). Fonts loaded eagerly only on desktop (`window.innerWidth >= 900`) or if already cached in `sessionStorage`. | Blocks LCP — kept minimal |
| **Lazy** | `loadLazy(doc)` | Remaining sections (`loadSections`), header, footer, `lazy-styles.css`, `fonts.css`, modal autolinking, hash-scroll, Sidekick quick-edit listeners. | Runs after first paint — safe for non-critical UI |
| **Delayed** | `loadDelayed()` | Dynamically imports `scripts/delayed.js` via `requestIdleCallback` (3s timeout fallback). Intended for analytics, martech, chat widgets. | Runs ~3s after load — never blocks rendering |

Rules for developers:

- Never add third-party scripts to `scripts.js` — they block LCP. Use `delayed.js`.
- `delayed.js` currently contains no integrations (placeholder comment only) — add analytics/martech here.
- Fonts are loaded lazily (or conditionally eager on desktop) to avoid render-blocking; never force eager font loading.
- The first section's blocks load eagerly; all others lazily, automatically based on DOM position.
- Header and footer load in the lazy phase via `loadHeader()` / `loadFooter()` from `aem.js`.
- `loadDelayed()` uses `requestIdleCallback` to avoid INP/TBT regressions.

## Key Files and How They Connect

| File | Role | Connects To |
|---|---|---|
| `scripts/aem.js` | Core AEM library — `loadBlock`, `loadCSS`, | Imported by `scripts.js` and most blocks |

| File | Role | Connects To |
|---|---|---|
| | `loadScript`, `decorateBlocks`, `decorateIcons`, `loadSection(s)`, `loadHeader`/`loadFooter`, `readBlockConfig`, `toClassName`, `getMetadata`, `createOptimizedPicture`, `waitForFirstImage`, `decorateTemplateAndTheme`, and the shared `DOMPURIFY` config. **DO NOT MODIFY.** | |
| `scripts/scripts.js` | Entry point. Orchestrates E-L-D, runs `decorateMain` (icons, auto-blocks, sections, blocks, buttons, a11y links). Exports project helpers: `moveInstrumentation` / `moveAttributes` (UE), `getBlockId`, `decorateButtons`, `decorateSections`, `ensureDOMPurify`, `NX_ORIGIN`. Handles section metadata including custom `background-color` / `background-image` decorations and `style` class mapping. | Imports from `aem.js`; imported by blocks for `moveInstrumentation`, `getBlockId`, `ensureDOMPurify` |
| `scripts/delayed.js` | Delayed-phase module (currently empty). Add analytics/marketing tags here. | Dynamically imported by `loadDelayed()` |
| `scripts/utils.js` | Shared YouTube/Vimeo embed-HTML builders (`getYoutubeEmbedHtml`, `getVimeoEmbedHtml`) with autoplay/background support. | Imported by `video` and `embed` blocks |
| `scripts/slider.js` | Shared slider engine — `createSliderControls`, `initSlider`, `showSlide`. | Imported by `carousel` and `card-carousel` blocks; `showSlide` re-exported for UE |
| `scripts/dompurify.min.js` | HTML sanitizer used to mitigate DOM XSS. Loaded once on demand via `ensureDOMPurify()`. | Used by `quote`, `table`, `video`, `embed`, `fragment` |
| `styles/styles.css` | LCP-critical CSS + design tokens (CSS custom properties on `:root`). Loaded eagerly. | Referenced by all blocks/pages |
| `styles/fonts.css` | `@font-face` declarations for Roboto. Loaded lazily. | Font families referenced in `styles.css` |
| `styles/lazy-styles.css` | Non-critical global styles, loaded lazily. | Supplements `styles.css` |
| `blocks/{name}/{name}.js` | Block logic — exports `default function decorate(block)`. Auto- | May import `aem.js`, `scripts.js` helpers, `utils.js`, `slider.js` |

| File | Role | Connects To |
|---|---|---|
| | loaded when the block class appears in the DOM. | |
| `blocks/{name}/{name}.css` | Block styles, auto-loaded with the block JS. Some blocks (e.g. `hero`) split tokens into `{name}-tokens.css`. | Uses tokens from `styles.css` |
| `ue/scripts/ue.js` | Universal Editor runtime support, dynamically imported in `scripts.js` only when host is `ue.da.live`. Handles tab/slide resync and UE instrumentation. | Imports `slider.js`, `ue-utils.js`, lazily `tabs.js` |

Execution flow: page load → `scripts.js` → (if `ue.da.live`, import `ue/scripts/ue.js`) → `loadPage()` → `loadEager()` (first section + LCP image) → `loadLazy()` (remaining sections, header, footer, lazy CSS, fonts) → `loadDelayed()` (imports `delayed.js`) → `loadSidekick()`.

### Universal Editor and Document Authoring Support

This project supports both authoring surfaces:

- **Document Authoring (DA)** — content authored on da.live; `scripts.js` includes DA-specific handling (e.g. `richtext` wrapping in `decorateSections`, the DA Sidekick loaded via `loadSidekick`, and DA NX preview/experiment hooks reading the `nx`/`dapreview`/`daexperiment` query params).
- **Universal Editor (UE)** — when the page is opened from `ue.da.live`, `scripts.js` dynamically imports `ue/scripts/ue.js` before `loadPage()`. The project exposes `moveInstrumentation(from, to)` (and `moveAttributes`) so blocks preserve `data-aue-*` / `data-richtext-*` instrumentation attributes when they rebuild DOM. Most blocks call `moveInstrumentation` during decoration so authors can edit them in-context.

UE relies on three generated JSON files at the repo root (`component-definition.json`, `component-models.json`, `component-filters.json`). These are NOT edited by hand — see "Universal Editor Models" below.

## Local Development Setup

### Prerequisites

- Node.js 24
- AEM CLI: `npm install -g @adobe/aem-cli`

### Setup Steps

```
git clone https://github.com/aemdemos/ise-boilerplate.git
cd ise-boilerplate
npm install        # also runs `playwright install chromium` via postinstall
aem up             # or: npx -y @adobe/aem-cli up --no-open --forward-browser-logs
```

- Local server runs at http://localhost:3000 with auto-reload.
- Inspect delivered HTML/DOM with `curl http://localhost:3000/{path}` (or append `.plain.html`).

### Test Content (without authored content)

Create static HTML files in a `drafts/` folder and start the dev server with `--html-folder drafts`. The `content/` folder in this repo contains several `*.plain.html` fixtures (e.g. `index.plain.html`, `test.plain.html`, `section-title-test.plain.html`) useful for local rendering.

### Linting

```
npm run lint        # runs lint:js then lint:css
npm run lint:js     # eslint . --ext .json,.js,.mjs
npm run lint:css    # stylelint blocks/**/*.css styles/*.css
npm run fix-js      # eslint --fix
npm run fix-css     # stylelint --fix
```

ESLint uses `eslint-config-airbnb-base` plus security/quality plugins (`sonarjs`, `secure-coding`, `browser-security`, `xwalk`). Stylelint uses `stylelint-config-standard`. Linting runs automatically on every PR via GitHub Actions and via a Husky pre-commit hook.

### Accessibility Tests

```
npm run test:a11y   # node tests/a11y/run.js (Playwright + @axe-core/playwright)
```

## Design System

The design system uses a two-tier token model:

- **Tier 1** — global semantic tokens declared in `:root` of `styles/styles.css` (the surface a theme/customer overrides).
- **Tier 2** — component tokens in `blocks/{name}/{name}-tokens.css` that reference Tier 1 tokens (e.g. `hero-tokens.css`).

Migration marker: the boilerplate carries a trailing `/* @boilerplate */` comment on every still-untouched CSS line. As styling is applied for a customer, each edited line's marker is deleted (never flipped). A Claude Code hook (`.claude/hooks/check-boilerplate-markers.mjs`) flags lines edited but left marked — but it stays silent in this source repo. Audit remaining work with:

```
grep -rn  '/* @boilerplate' styles/ blocks/   # every untouched line
grep -rc  '/* @boilerplate' styles/ blocks/   # remaining count per file
```

### CSS Custom Properties (Tokens)

Colors:

```
--background-color: white;
--light-color: #f8f8f8;
--dark-color: #505050;
--text-color: #131313;
--link-color: #3b63fb;
--link-hover-color: #1d3ecf;
--overlay-background-color: lightgrey;
--error-color: firebrick;
/* semantic aliases */
--color-surface: var(--background-color);
```

```
--color-on-surface: var(--text-color);
--color-action: var(--link-color);
--color-action-hover: var(--link-hover-color);
```

Typography:

```
--body-font-family: roboto, roboto-fallback, sans-serif;
--heading-font-family: roboto-condensed, roboto-condensed-fallback, sans-serif;
--fixed-font-family: 'Roboto Mono', menlo, consolas, monospace;
--body-line-height: 1.6;
--heading-line-height: 1.25;
--heading-font-weight: 600;
```

Type scale (mobile base; reduced at >= 900px):

| Token | Mobile | Desktop (>= 900px) |
|---|---|---|
| --body-font-size-m | 22px | 18px |
| --body-font-size-s | 19px | 16px |
| --body-font-size-xs | 17px | 14px |
| --heading-font-size-xxl (h1) | 55px | 45px |
| --heading-font-size-xl (h2) | 44px | 36px |
| --heading-font-size-l (h3) | 34px | 28px |
| --heading-font-size-m (h4) | 27px | 22px |
| --heading-font-size-s (h5) | 24px | 20px |
| --heading-font-size-xs (h6) | 22px | 18px |

Spacing, layout, borders, motion:

```
--spacing-xs: 8px;  --spacing-s: 16px;  --spacing-m: 24px;
--spacing-l: 32px;  --spacing-xl: 40px;
--max-width-site: 1200px;   --icon-size: 24px;
--nav-height: 64px;  --breadcrumbs-height: 34px;  --header-height: var(--nav-height);
--border-width-s: 1px;  --border-color: #dadada;
--border-radius-s: 4px;  --border-radius-m: 8px;
--transition-duration: 0.2s;
--block-margin-top: 0.8em;  --block-margin-bottom: 0.25em;
--focus-ring-width: 2px;  --focus-ring-offset: 2px;  --focus-ring-color: var(--link-
color);
```

Button tokens (`--button-*`) drive the global `a.button` / `button` styling. `decorateButtons()` in `scripts.js` maps author formatting to variants: `**bold link**` → `.button.primary`, `*italic link*` → `.button.secondary`, `***bold italic***` → `.button.accent`.

## Fonts

Roboto is the project font, self-hosted as woff2 in `fonts/` and declared in `styles/fonts.css`:

| Family | Weights | Source file | Usage |
|---|---|---|---|
| roboto | 400, 500, 700 | roboto-regular.woff2, roboto-medium.woff2, roboto-bold.woff2 | Body text |
| roboto-condensed | 700 | roboto-condensed-bold.woff2 | Headings |

Fallback @font-face rules in styles.css (roboto-fallback, roboto-condensed-fallback) use size-adjust against local Arial to minimize CLS. All faces use font-display: swap.

### Breakpoints

Mobile-first, min-width only. The primary breakpoint in this project is **900px** (desktop).

| Name | Min-Width | Usage |
|---|---|---|
| Mobile | 0 (default) | Base styles |
| Tablet | 600px | @media (width >= 600px) |
| Desktop | 900px | @media (width >= 900px) — type-scale shift, section/nav layout |
| Wide | 1200px | --max-width-site content cap |

### Section Styles

Section styling is driven by section metadata (style field) and decorated in decorateSections():

- **light** and **highlight** — both apply background-color: var(--light-color) with --spacing-xl vertical padding (see styles.css).
- **Background color / image** — sections support background-color (validated hex/rgb/name/token) and background-image (http(s) only) metadata; the image is injected as a decorative .bg-image layer behind the content. Values are sanitized in scripts.js to prevent CSS/URL injection (CWE-770/915 guards).
- Any other comma-separated style values are converted to classes via toClassName and added to the section element.

## Blocks Reference

Twenty blocks ship with the project. Many call getBlockId() to assign unique ids (for ARIA aria-controls/aria-labelledby and martech tracking) and moveInstrumentation() to preserve UE editing attributes. Blocks needing HTML sanitization call ensureDOMPurify().

| Block | Purpose | Variants / Options | Key Details |
|---|---|---|---|
| accordion | Expand/collapse list of label + body rows | — | Rebuilds rows into <ul> of .accordion-item; toggles .active on label click; preserves UE instrumentation |
| card | Shared single-card builder (not authored directly) | — | Exports createCard(row) producing a card <li> with .cards-card-image / .cards-card-body; reused by cards and card-carousel |
| cards | Grid of cards | — | Builds <ul> of cards; optimizes images via |

| Block | Purpose | Variants / Options | Key Details |
|---|---|---|---|
| | | | `createOptimizedPicture` (750px); ARIA region |
| `card-carousel` | Horizontally sliding cards | single-slide auto-detected | Uses shared `slider.js` (`createSliderControls`, `initSlider`, `showSlide`); reuses `createCard` |
| `carousel` | Full-width slide carousel (image + content) | single-slide auto-detected | Uses `slider.js`; each slide split into `.carousel-slide-image` / `.carousel-slide-content`; sets `aria-labelledby` from heading |
| `columns` | Multi-column layout | auto class `columns-{N}-cols` | Adds `.columns-img-col` to image-only columns; ARIA region |
| `embed` | Embeds videos/social posts | `embed-youtube`, `embed-vimeo` (auto from URL), `embed-is-loaded`, `embed-placeholder` | Lazy-loads on intersection; uses `utils.js` embed builders; default iframe wrapper for other hosts |
| `footer` | Site footer | — | Loads footer fragment from `footer` metadata path (default `/footer`) via `loadFragment` |
| `form` | Renders a form from a JSON definition | — | Fetches form JSON, builds fields via `form-fields.js`, groups `fieldsets`, sets `data-action` submit href |
| `fragment` | Inline another document as content | — | `loadFragment(path)` fetches `.plain.html`, sanitizes via DOMPurify, copies section classes onto host; also drives auto-blocking of `/fragments/` links |
| `header` | Site navigation / header | `nav-drop` (dropdown sections), `nav-hamburger` (mobile) | Loads nav fragment from `nav` metadata; desktop/mobile behavior via `matchMedia(900px)`; ESC + focus-out close handling |
| `hero` | Full-bleed hero with background image | CSS-only block | No JS (`hero.js` is empty); styled via `hero.css` + `hero-tokens.css`; picture positioned behind heading |
| `modal` | Modal dialog | — | No `decorate`; links to `/modals/` auto-open via `autolinkModals` in `scripts.js`; exports `createModal`/`openModal`; uses native `<dialog>` |
| `quote` | Blockquote with attribution | — | `.quote-quotation` + `.quote-attribution`; wraps `<em>` attribution in `<cite>` (DOMPurify-sanitized); ARIA region |
| `search` | Client-side site search | `no-results` state | Reads search index, highlights matching terms, optimizes result images, computes heading level dynamically |

| Block | Purpose | Variants / Options | Key Details |
|---|---|---|---|
| `section-title` | Heading + optional subtitle | size `size-xs..xxl`, alignment `left/center/right`, color `section-title-color-{background,light,dark,text,link,link-hover}`, subtitle `subtitle-size-*` | Supports legacy 4-row table and UE/DA key-value rows; color keys map to `:root` tokens |
| `table` | Renders a table | `no-header` | Builds `<table>` with optional `<thead>`; cells sanitized via DOMPurify; ARIA region |
| `tabs` | Tabbed panels | — | Tablist with delegated click handling, `aria-controls/aria-selected`; UE resync via `ue.js`; container block (`tabs-item` children) |
| `video` | Inline / background video | `autoplay`, `placeholder` (auto), background mode | Supports MP4, YouTube, Vimeo (via `utils.js`); respects `prefers-reduced-motion`; click-to-play placeholder |

## Universal Editor Models

The three root JSON files consumed by Universal Editor are **generated**, not hand-edited:

- `component-definition.json`
- `component-models.json`
- `component-filters.json`

### Source of Truth: `ue/models/`

Edit the fragments under `ue/models/` and regenerate. The directory contains:

- Top-level fragments: `component-definition.json`, `component-models.json`, `component-filters.json`, plus base models `page.json`, `section.json`, `text.json`, `image.json`.
- `ue/models/blocks/` — one JSON per editable block (accordion, card, card-carousel, cards, carousel, columns, fragment, hero, quote, search, section-title, table, tabs, video). Each may declare `definitions`, `models`, and `filters` (filters used for container blocks such as `tabs`).

### Build Process

`merge-json-cli` merges the fragments into the three root files:

```
npm run build:json
```

This runs three merges in parallel (`npm-run-all -p`):

```
npm run build:json:models       # ue/models/component-models.json      -> component-models.json
npm run build:json:definitions  # ue/models/component-definition.json   -> component-definition.json
npm run build:json:filters      # ue/models/component-filters.json      -> component-filters.json
```

Workflow to add/modify a UE-editable block:

1. Add the block to a test page and open it in Universal Editor.

2. Inspect the `/details` network call to understand the structure.
3. Create or edit the block's fragment in `ue/models/blocks/{block}.json` (definition, model, and filter if it is a container).
4. Register the block in `ue/models/section.json`'s filter list.
5. Run `npm run build:json` and commit both the `ue/models/` source and the regenerated root JSON.

For block options, use select components named `classes` or `classes_{suffix}`; they are combined into the block's class list at render time.

---

# Common Development Tasks

## Add a New Block

1. Create `blocks/{name}/{name}.js` exporting `default function decorate(block)`.
2. Create `blocks/{name}/{name}.css` with all selectors scoped to `.{name}`.
3. Read the DOM the backend delivers, transform it in place, preserve UE instrumentation with `moveInstrumentation` if you rebuild nodes.
4. If the block needs UE editing, add `ue/models/blocks/{name}.json`, register it in `ue/models/section.json`, and run `npm run build:json`.
5. Test locally at http://localhost:3000.

## Modify Global Styles

1. Edit `styles/styles.css` (tokens) or `styles/lazy-styles.css` (non-critical).
2. Use `var(--token)` values; remove the `/* @boilerplate */` marker on any line you intentionally change.
3. Test across multiple pages and watch for CLS.

## Add Analytics / Marketing Tool

1. Add the integration to `scripts/delayed.js` (currently empty).
2. Never add third-party scripts to `scripts.js` — it blocks LCP.
3. Verify in the Network tab that it loads ~3s after page load.

## Debug a Block

1. Check the browser console for errors.
2. Compare the expected vs actual DOM (`console.log(block.innerHTML)`).
3. Confirm variant classes are applied (e.g. `embed-youtube`, `no-header`, `columns-3-cols`).
4. Check CSS specificity and that the block CSS file loaded (Network tab).

---

# Environments

| Environment | URL Pattern | Purpose |
|---|---|---|
| Local | http://localhost:3000 | Development |
| Feature Branch | https://{branch}–ise-boilerplate–aemdemos.aem.page/ | PR testing |
| Preview | https://main–ise-boilerplate–aemdemos.aem.page/ | Staging |

| Environment | URL Pattern | Purpose |
|---|---|---|
| Live | https://main--ise-boilerplate--aemdemos.aem.live/ | Production |

URL parts: `{repo}` = `ise-boilerplate`, `{owner}` = `aemdemos`; `{branch}` from `git branch`.

## Git and Release Workflow

- Branch naming: `feature/description`, `fix/description` (keep short for URL limits).
- Commits follow Conventional Commits — releases are automated with `semantic-release` (`.releaserc.cjs`); `npm run release` runs in CI.
- Husky pre-commit hook runs linting; CI runs `npm run lint` on every PR.
- PR requirements: include before/after preview URLs, pass linting, no console errors, PageSpeed Performance must score 100 on the preview URL. Run `gh pr checks` before requesting review.

## Troubleshooting

### Block not loading

1. Confirm the block folder name matches the authored block class name.
2. Verify the JS exports `default function decorate(block)` (note: `modal` and `card` are intentional exceptions — modal auto-links, card exports `createCard`).
3. Check the Network tab for 404s on `blocks/{name}/{name}.js` / `.css`.

### Styles not applying

1. Check CSS specificity and that selectors are scoped to the block.
2. Confirm the file loaded (Network tab).
3. Look for stray `/* @boilerplate */`-marked lines you meant to change.

### Universal Editor changes not appearing

1. Did you edit `ue/models/` and run `npm run build:json`? The root JSON files are generated.
2. Confirm the block is registered in `ue/models/section.json`'s filter list.
3. Ensure the block preserves instrumentation with `moveInstrumentation` when it rebuilds DOM.

### Content not updating

1. Clear browser cache.
2. Confirm preview was triggered in Document Authoring / Sidekick.
3. Allow 1-2 minutes for CDN cache.

### Sanitization / embed issues

- `video`, `embed`, `quote`, `table`, and `fragment` depend on DOMPurify via `ensureDOMPurify()`; if content renders empty, check the console for sanitizer-stripped markup.
- YouTube/Vimeo logic lives in `scripts/utils.js` — autoplay and background modes are controlled by the `autoplay` block class and URL parameters.

## Resources

| Resource | URL |
|---|---|
| AEM Edge Delivery Services Docs | https://www.aem.live/docs/ |
| Developer / UE Tutorial | https://www.aem.live/developer/ue-tutorial |
| Universal Editor Blocks | https://www.aem.live/developer/universal-editor-blocks |
| Component Model Definitions | https://www.aem.live/developer/component-model-definitions |
| Block Collection | https://www.aem.live/developer/block-collection |
| Keeping It 100 (E-L-D / performance) | https://www.aem.live/developer/keeping-it-100 |
| Markup, Sections, Blocks, Auto Blocking | https://www.aem.live/developer/markup-sections-blocks |
| Universal Editor (da-live wiki) | https://github.com/adobe/da-live/wiki/Universal-Editor |